

Pathfinding em Malhas 3D

Análise estatística e comparativa

André Vitor Bastos de Macêdo
Orientador: Prof. Paulo César Rodacki Gomes

Instituto Federal Catarinense
Campus Blumenau

2026



INSTITUTO FEDERAL
Catarinense
Campus Blumenau

1. Introdução
2. Objetivos
3. Justificativa
4. Trabalhos Relacionados
5. Referencial Teórico
6. Metodologia
7. Resultados e Próximos Passos
8. Conclusão
9. Referências

Introdução e Definição do Problema

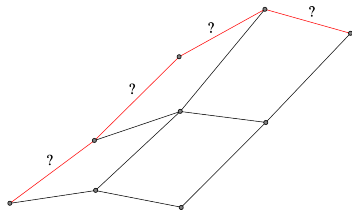
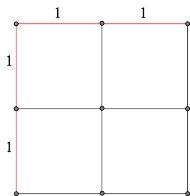
Busca de caminhos (ou *pathfinding*) é a lógica de encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração de um grafo [2].

De puramente abstratos para aplicação direta na computação interativa, é um componente crucial para o desenvolvimento de jogos e simulações em tempo real [11].

Introdução e Definição do Problema

Em espaços 2D, a passagem entre coordenadas inteiras é uniforme e fácil de se navegar.

No espaço 3D, é preciso considerar a variação de altura, inclinação do relevo e obstáculos complexos. Isso traz instabilidade e inconsistências matemáticas nos cálculos de distância [14].



Geral: Analisar comparativa e estatisticamente o desempenho de algoritmos de busca altamente usados em múltiplos cenários 3D.

Específicos:

- ➊ Implementar uma biblioteca de grafos eficiente para execução de algoritmos de pathfinding.
- ➋ Desenvolver uma infraestrutura de renderização para visualização das malhas e caminhos encontrados.
- ➌ Adaptar a arquitetura para execução concorrente e testes de estresse.
- ➍ Desenvolver geradores procedurais de malhas para diferentes cenários de teste.
- ➎ Analisar estatisticamente os resultados obtidos nos testes.

Existe uma grande lacuna em estudos de busca de caminhos na área de renderização 3D. Pesquisas recentes tendem a focar unicamente em ambientes 2D ou em um único modelo de interpretação de malhas, como Johansson (2024)[5] e Kapi (2022)[6]. Este estudo visa superar essa barreira e comparar diferentes estilos de modelagem de terreno.

Adicionalmente, existe a necessidade de separar a análise comparativa dos grandes motores gráficos (Unity, Unreal e outras) e trazer uma visão única e sem ruídos em uma engine criada do zero. Isso nos permite medir a performance de CPU e memória de forma pura.

Trabalhos Relacionados e Diferencial Técnico

Table: Matriz de diferenciação técnica

Crítérios	Proposta	Johansson (2024)[5]	Kapi (2022)[6]
Dimensão Espacial	Malhas 3D volumétricas, Octrees e Voxels	Voxels (Blocos 3D)	Grades 2D planas
Geometria	Procedural	Cubos fixos	Matrizes planas
Algoritmos	Dijkstra, A*, JPS e Theta*	A* e Dijkstra	A*, JPS e Theta*
Arquitetura	Paralela	Sequencial	Sequencial
Ambiente/Carga	Renderização 3D concorrente	Execução isolada	Execução isolada

Esta pesquisa se baseia em conceitos de grafos, algoritmos de busca, geração procedural de malhas 3D e técnicas de paralelização. A seguir, serão detalhados os principais conceitos que fundamentam o desenvolvimento do projeto.

Definição matemática [2]:

$$G = (V, A) \quad (1)$$

Onde:

- G é um grafo.
- V é um conjunto de vértices (entidades individuais, ex: coordenadas).
- A é um conjunto de arestas (ligações direcionadas ou não que carregam pesos/custos).

Um algoritmo de busca é o procedimento que atravessa os vértices do grafo a partir das arestas e, com seus respectivos pesos, encontra o caminho de menor custo entre dois vértices.

O algoritmo de Dijkstra [2] é o algoritmo fundamental de caminho mínimo e o “pai” dos principais algoritmos da atualidade. Com a adição de heurísticas ao seu processo, outros algoritmos como A*, JPS e Theta* foram desenvolvidos.

Enquanto o Dijkstra verifica apenas o peso real das arestas, os demais se baseiam na soma do peso e o valor heurístico $f(n) = g(n) + h(n)$.

- **Dijkstra:** busca exaustiva.
- **A*:** uso de heurísticas espaciais [3].
- **JPS:** otimização para “saltar” blocos vazios [10].
- **Theta*:** busca em qualquer ângulo, suaviza o caminho [9].

Exemplos de heurísticas espaciais:

- **Manhattan:**

$$h(n) = |x_1 - x_2| + |y_1 - y_2| \quad (2)$$

- **Euclidiana (3D):**

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (3)$$

- **Diagonal (Chebyshev):**

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|) \quad (4)$$

Representação de Malhas 3D

Uma malha 3D (ou poligonal) é a forma de um objeto em um espaço tridimensional [15]. Elas são compostas por:

- **Vértices:** Os pontos no espaço 3D, definidos por (x, y, z) .
- **Arestas:** As linhas retas que conectam dois vértices.
- **Faces:** Os polígonos delimitados por arestas.

Já evidenciando sua possível relação com grafos...

Malhas 3D no Motor IFCG

Demonstração conceitual da malha:

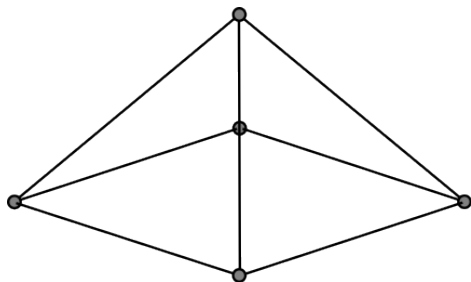
- **Vértices** demarcados por coordenadas espaciais.
- **Arestas** conectando os nós lógicos.
- **Faces** trianguladas prontas para renderização.
- **Malha** finalizada com preenchimento.



Malhas 3D no Motor IFCG

Demonstração conceitual da malha:

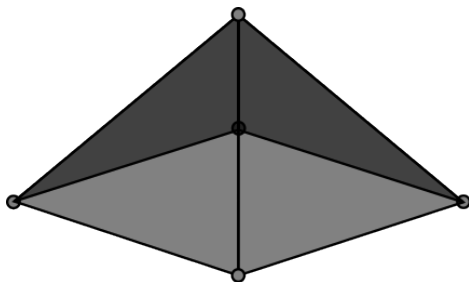
- **Vértices** demarcados por coordenadas espaciais.
- **Arestas** conectando os nós lógicos.
- Faces trianguladas prontas para renderização.
- **Malha** finalizada com preenchimento.



Malhas 3D no Motor IFCG

Demonstração conceitual da malha:

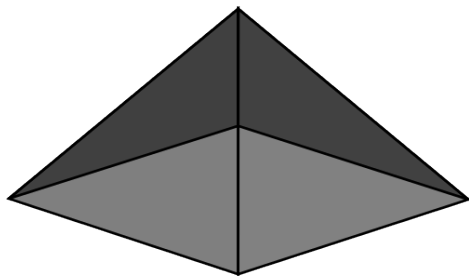
- **Vértices** demarcados por coordenadas espaciais.
- **Arestas** conectando os nós lógicos.
- **Faces** trianguladas prontas para renderização.
- **Malha** finalizada com preenchimento.



Malhas 3D no Motor IFCG

Demonstração conceitual da malha:

- **Vértices** demarcados por coordenadas espaciais.
- **Arestas** conectando os nós lógicos.
- **Faces** trianguladas prontas para renderização.
- **Malha** finalizada com preenchimento.



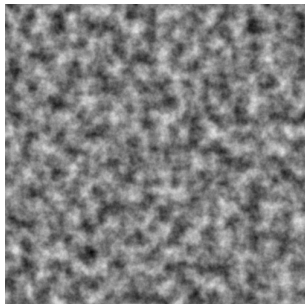
Para garantir uma grande quantidade de cenários de teste, foi utilizado Ruído de Perlin [13] para gerar de forma automatizada as malhas de teste [12]. O algoritmo funciona empilhando múltiplas camadas ligeiramente diferentes (oitavas) de ruído para formar uma imagem contínua e suave.

Em uma oitava, o espaço da imagem é dividido em uma grade e em cada interseção é atribuído um vetor de gradiente aleatório [17]. Em cada pixel da imagem, é calculado o vetor que vai dos nós da grade até esse ponto e é feita uma multiplicação escalar. Os valores calculados nos nós vizinhos são interpolados e finalmente um valor é atribuído ao pixel.

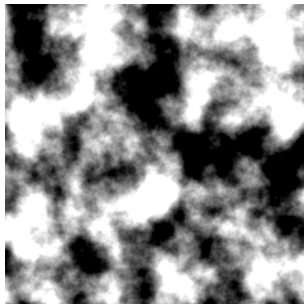
Geração Procedural

Para que as oitavas não sejam idênticas, valores de lacunaridade (frequência) e persistência (amplitude) influenciam no cálculo dos valores finais dos pixels [12]. A frequência define a quantidade de detalhes, enquanto a amplitude define a escala de intensidade desses detalhes.

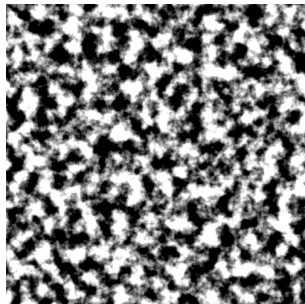
Ruído de alta frequência



Ruído de alta amplitude

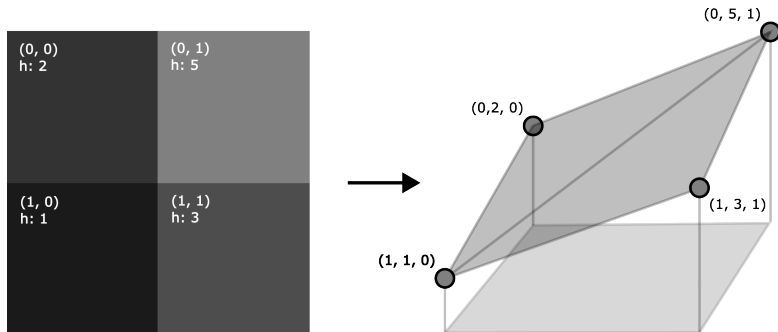


Ruído de alta freq. e amp.



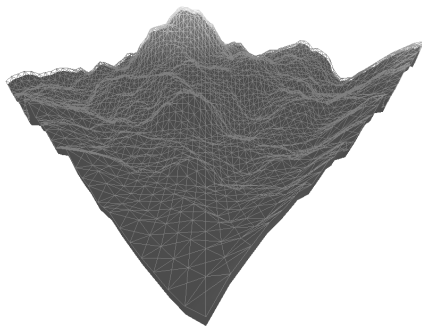
Conversão Pixel para Vértice

Com uma imagem de ruído gerada, traduzem-se os valores para malha e grafo, mapeando a posição (x, z) dos pixels aos vértices e convertendo o valor da cor para a altura.



Malha Resultante

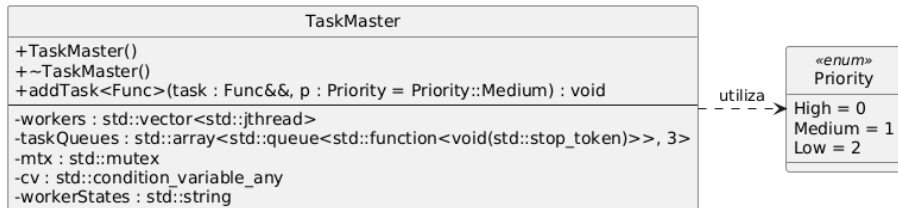
Exemplo de malha gerada a partir de uma imagem de ruído de Perlin.



Paralelização

Todo o ambiente de teste e renderização será paralelizado pela classe TaskMaster. Ela encapsula conceitos de Monitor, Filas Multinível [7] e Task Scheduler [16].

Essa arquitetura garante um ambiente concorrente limpo e seguro, executando testes enquanto o motor gráfico roda em segundo plano.



Os algoritmos serão testados em diferentes cenários de teste, alternando entre possíveis configurações de ruídos:

- **Escala:** Tamanho da malha ($N \times N$).
- **Lacunaridade:** Quantidade de obstáculos.
- **Persistência:** Nível de dificuldade dos obstáculos.

Em todos os testes serão testados:

- Tempo de CPU.
- Consumo de memória.
- Qualidade do caminho (custo).
- Taxa de erro de cache.

Será priorizado um ambiente quiescente para não haver interferências do sistema operacional [4, 8].

Riscos Associados ao Projeto

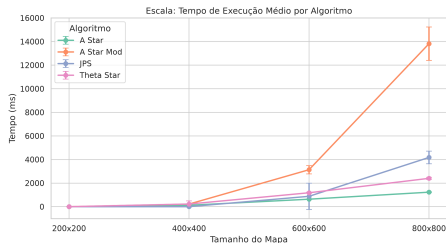
Table: Riscos do projeto e estratégias de mitigação

Risco	Mitigação
Complexidade de adaptar algoritmos complexos como JPS 3D e Theta*	Referências de alto valor, uso de bibliotecas de teste (GTest) e otimizações de compilação
Coleta de amostras e viés de dados	Controle rígido do ambiente de teste e forte embasamento teórico
Expansão ambiciosa e insuficiência do tempo planejado	Cronograma modular, priorização de adições e ambiente de pesquisa estável

Desenvolvimento e Resultados Parciais

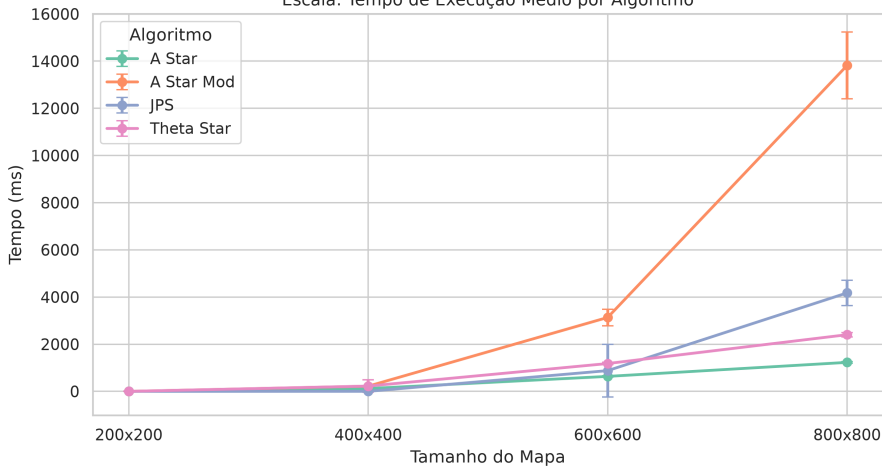
Análise do Tempo de Execução (ms):

- **A* Modificado:** Apresentou o pior desempenho em grandes dimensões, atingindo 13816.9 ms em malhas de 800×800 vértices.
- **JPS:** Mais eficiente em escalas menores, porém degradou nas maiores (4175.0 ms em 800×800).
- **A* e Theta*:** Mantiveram-se mais consistentes ao longo do crescimento da escala.



Resultados Parciais

Escala: Tempo de Execução Médio por Algoritmo



Resultados Esperados

- Distinguir melhores algoritmos para cada modelo geométrico e tipo de cenário:
 - Dijkstra com pior tempo de execução (baseline).
 - JPS com melhor tempo em áreas com menos obstáculos.
 - Theta* com melhor qualidade de caminhos.
- Validar a influência da escala e dificuldade de terreno no tempo de execução de algoritmos de busca:
 - ANOVA com $p\text{-valor} < 0,05$ e teste complementar de Tukey [1].

Table: Cronograma de execução das atividades

Etapa	Período	Status
Revisão Bibliográfica	Jan-Fev/2026	Concluído
Motor Gráfico e Grafos	Fev-Abr/2026	Concluído
Algoritmos de Busca	Mar-Mai/2026	Concluído
Geração de Cenários	Abr-Mai/2026	Concluído
Concorrência	Mai-Jun/2026	Concluído
Planejamento da coleta de dados	Mai-Jun/2026	Concluído
Defesa do Pré-Projeto	Jun/2026	Concluído
NavMeshes, Octrees e Voxels	Jul-Set/2026	Pendente
Testes de Estresse	Ago/2026	Pendente
Algoritmos Adicionais	Ago-Set/2026	Pendente
Coleta Automatizada de Dados	Set-Out/2026	Pendente
Análise estatística	Out-Nov/2026	Pendente
Redação Final da Monografia	Out-Dez/2026	Pendente
Defesa do TCC	Dez/2026	Pendente

Considerações Finais e Conclusão

A elaboração do pré-projeto consolidou uma base técnica estável e demonstrou a viabilidade da pesquisa comparativa de pathfinding 3D concorrente.

Principais conclusões do estágio atual:

- **Viabilidade técnica confirmada:** Motor gráfico IFCG e estruturas concorrentes (TaskMaster) funcionando sem instabilidades.
- **Rigor estatístico:** A infraestrutura e o isolamento de CPU garantem medições precisas para ANOVA.
- **Contribuição prática:** Desenvolvimento open-source de bibliotecas de grafos leves.

Referências I

- [1] Jurandir Junior Domingues. *Material didático*. Notas de aula, Instituto Federal Catarinense (IFC). Blumenau, SC, 2026.
- [2] Paulo César Rodacki Gomes. *Grafos: conceitos fundamentais, algoritmos e aplicações*. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022. ISBN: 978-65-88089-12-5.
- [3] Peter E Hart, Nils J Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
- [4] Raj Jain. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: John Wiley & Sons, 1991. ISBN: 978-0471503361.
- [5] E. Johansson et al. "Adapting Pathfinding Algorithms for 3D Environments: Performance Analysis and Real-World Applications". Estudo comparativo de memória e tempo de CPU em malhas 3D discretas. MA thesis. Linnæus University, 2024.
- [6] Azyan Yusra Kapi, Mohd Shahrizal Sunar, and Zeyad Abd Algfoor. "A Comparison of Pathfinding Algorithm for Code Optimization on Grid Maps". In: *International Journal of Advanced Computer Science and Applications* 13.12 (2022). DOI: 10.14569/IJACSA.2022.0131247. URL: <http://dx.doi.org/10.14569/IJACSA.2022.0131247>.
- [7] Ricardo de la Rocha Ladeira. *Material didático*. Notas de aula, Instituto Federal Catarinense (IFC). Blumenau, SC, 2026.
- [8] David J. Lilja. *Measuring Computer Performance: A Practitioner's Guide*. Cambridge: Cambridge University Press, 2000. ISBN: 978-0521641074.
- [9] Alex Nash and Sven Koenig. "Any-Angle Path Planning". In: *AI Magazine* 34.4 (2013), pp. 85–107.
- [10] Thomas K. Nobes et al. "The JPS Pathfinding System in 3D". In: *Proceedings of the International Symposium on Combinatorial Search* 15.1 (July 2022), pp. 145–152. DOI: 10.1609/socs.v15i1.21762. URL: <https://ojs.aaai.org/index.php/SOCS/article/view/21762>.
- [11] Sara Lutami Pardede et al. "A review of pathfinding in game development". In: *Journal of Computer Engineering* 1.01 (2022), pp. 47–56.

- [12] Amit J. Patel. *Making maps with noise functions*. Acessado em: 07 mai. 2026. Red Blob Games. 2015. URL: <https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution> (visited on 05/07/2026).
- [13] Ken Perlin. "An Image Synthesizer". In: *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '85. New York, NY, USA: Association for Computing Machinery, 1985, pp. 287–296. ISBN: 0897911660. DOI: 10.1145/325334.325247. URL: <https://doi.org/10.1145/325334.325247>.
- [14] Jakub Smotka et al. "A* pathfinding algorithm modification for a 3D engine". In: *MATEC Web Conf.* 252 (2019), p. 03007. DOI: 10.1051/mateconf/201925203007. URL: <https://doi.org/10.1051/mateconf/201925203007>.
- [15] Joey de Vries. *Learn OpenGL: Enjoy learning modern OpenGL*. Acessado em: 2026-04-18. 2014. URL: <https://learnopengl.com/> (visited on 04/18/2026).
- [16] Anthony Williams. *C++ Concurrency in Action*. 2nd. Manning Publications, 2019. ISBN: 978-1617294693.
- [17] Zipped. *C++: Perlin Noise Tutorial*. Acessado em: 07 mai. 2026. YouTube. July 2023. URL: <https://www.youtube.com/watch?v=kCIaHqb60Cw> (visited on 05/07/2026).

Pathfinding em Malhas 3D

Análise estatística e comparativa

André Vitor Bastos de Macêdo
Orientador: Prof. Paulo César Rodacki Gomes

Instituto Federal Catarinense
Campus Blumenau

2026



INSTITUTO FEDERAL
Catarinense
Campus Blumenau